# Lesson 02 · C# Language Essentials

Syntax, types, nullability, pattern matching, methods and exceptions — read against Kotlin, Java 21 and TypeScript, with the Visual Basic form beside it

## Contents


**1 · Why this lesson — the syntax you will review tomorrow** 2
- 1.1 · The numbers to leave this lesson with 2

**2 · The shape of a C# file** 2
- 2.1 · Syntax map — Kotlin, Java and TypeScript → C# 3
- 2.2 · An entry point — Kotlin main() vs C# top-level statements 3
- 2.3 · Two more files — global usings, and a namespace with no imports 4
- 2.4 · A file-based app — the whole program, no project file 4
- 2.5 · What the compiler is given — a project build and a file-based run 5
- 2.6 · Date a codebase by its syntax — this lesson's C# by version 5

**3 · Types and values — decimal, strings and culture** 5
- 3.1 · Money — Java BigDecimal vs C# decimal 6
- 3.2 · Rounding — Java's explicit mode vs C#'s silent default 6
- 3.3 · Aliases, var, const and overflow 6
- 3.4 · The built-in types you will use every day 7
- 3.5 · Interpolation, verbatim and raw string literals 8
- 3.6 · Comparison and culture — the traps with measured output 8

**4 · Nullability — annotations, not a type system** 8
- 4.1 · The null operators — Kotlin vs C# 9
- 4.2 · What survives to run time 9
- 4.3 · Null-state analysis — what the compiler tracks for one variable 10
- 4.4 · Kotlin guarantees vs C# promises 10

**5 · Control flow and pattern matching** 10
- 5.1 · A rating rule — Kotlin when vs a C# switch expression 10
- 5.2 · The enum trap — a complete switch that still needs its discard arm 11
- 5.3 · Property and tuple patterns in the rating rules 11
- 5.4 · Type, property and declaration patterns on an object 12
- 5.5 · List patterns — parsing a partner rate file 12
- 5.6 · Pattern kinds — the C# form, its version and your nearest equivalent 12
- 5.7 · Pattern-matching support across the languages you read 13

**6 · Methods and parameters** 13
- 6.1 · Parameter passing at a glance 13
- 6.2 · Declaring methods — TypeScript vs C# 14
- 6.3 · Calling them — named arguments, deconstruction, out var and a C# 14 lambda 14

**7 · Exceptions and resources** 14
- 7.1 · Resources and filtered catch — Java vs C# 15
- 7.2 · When does a filter run? Proven by the sample 15
- 7.3 · Two-pass exception handling 16

**8 · Visual Basic alongside** 16
- 8.1 · The age loading — C# switch expression vs VB Select Case 16
- 8.2 · Resources and a filtered catch — C# using var vs VB Using … End Using 17
- 8.3 · Overflow — the one default VB reverses 17
- 8.4 · This lesson's C# in Visual Basic 17

**9 · Hands-on — run the tour, the tests and the file-based app** 18
- 9.1 · Commands 18
- 9.2 · What you should see — the C# and VB tours 18
- 9.3 · The test run and the file-based app 19
- 9.4 · The premium calculation — the code behind the worked example 19
- 9.5 · The worked example — how 8,933.71 THB is built 19
- 9.6 · The same behaviour, C# vs VB lines 20
- 9.7 · Test cases per tested member 20


Page numbers are resolved from the rendered PDF. Every C# and VB panel is cut from a sample that builds, tests and runs on the .NET 10 SDK.

## 1 · Why this lesson — the syntax you will review tomorrow

**You already think in every idea this lesson covers; what you lack is the C# spelling and the places where the spelling lies.** Null safety, pattern matching, value objects for money, resource scopes and exception handling are things you have shipped in Kotlin, Java and TypeScript. C# has all of them — several with the same symbols — and a few of those identical-looking symbols mean something different. A reviewer who reads `string?` as Kotlin's `String?`, or `!` as `!!`, approves code that fails in production.

**The language moved fast, so the age of the syntax dates the code.** The C# in a 2016 service and in a .NET 10 service can look like two languages: top-level statements, file-scoped namespaces, switch expressions, nullable reference types, raw strings and list patterns all arrived between C# 6 (2015) and C# 14 (November 2025, with .NET 10). This lesson teaches the current form and names the version of each feature (table 2.6, dated from The history of C#), so you can read an older estate too.

**Everything is taught through a rating engine, because that is where syntax bugs cost money.** The samples calculate a motor premium — base rate by coverage class, loadings for young drivers, claims and commercial use, a no-claim bonus, stamp duty and VAT. The rates are **illustrative, not a real tariff** — and they are the one tariff the whole track prices with, so the totals in §9 recur in every later lesson. A culture that swaps a decimal separator, a rounding mode that moves a satang, a year printed in the Buddhist era: each one is demonstrated by a sample that ran, not described.

### 1.1 · The numbers to leave this lesson with

| C# 14 | DECIMAL DIGITS | PATTERN KINDS | RATING TEST CASES | LESSON 02 SAMPLES |
|---|---|---|---|---|
| **Nov 2025** | **28–29** | **10** | **31** | **690 lines** |
| ships with .NET 10 · verified | significant · 16 bytes, base 10 · verified | listed in the C# reference · verified | xUnit, all passing · measured at build | 545 C# + 145 VB · measured |

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# VB Java/Kotlin TypeScript

### i Before you start

1. **Prerequisite:** lesson 01 (The .NET Platform Map) — the .NET 10 SDK first on PATH, and the idea that C# and VB compile to the same IL. `dotnet --list-sdks` must list 10.0.401 or a later 10.0 SDK.
2. **You will build:** `L02.Rating` (a C# library with the rating rules), `L02.Rating.Tests` (31 xUnit test cases), `L02.Syntax` and `L02.SyntaxVb` (the same language tour in C# and Visual Basic), and a .NET 10 file-based app with no project file.
3. **Not here:** classes, records and generics are lesson 03 (Types, OOP & Generics); LINQ and lambdas lesson 04 (Collections, LINQ & Functional C#); async lesson 05 (Async, Tasks & Concurrency); VB semantics in depth lesson 06 (VB.NET for Legacy Estates). Where a sample uses one of them, read it as vocabulary.
4. Facts that change between releases are marked • Verified and linked; figures computed from the samples are • Measured; judgement calls are • Estimate. Kotlin, Java and TypeScript panels are for comparison and are not compiled.

## 2 · The shape of a C# file

**A modern C# file has less ceremony than a Java file and roughly as little as a Kotlin file.** One file per project may hold *top-level statements* — that file is the entry point, with no class and no `Main`. Every other file starts with a *file-scoped namespace* (`namespace L02.Syntax;`, C# 10), so the type body is not indented inside braces. A namespace is a naming convention the compiler does not tie to folders — Java expects a package's classes in a matching directory; C# does not.

**Most imports are not in the file at all.** With `ImplicitUsings` on, the SDK generates `global using` directives for `System`, `System.Collections.Generic`, `System.IO`, `System.Linq`, `System.Net.Http`, `System.Threading` and `System.Threading.Tasks`; a project adds its own with a `<Using>` item or a `global using` line (.NET project SDK overview). When a reviewed file uses `CultureInfo` with no import in sight, the import is in a generated `<ProjectName>.GlobalUsings.g.cs` under `obj` — here `L02.Syntax.GlobalUsings.g.cs` in `.build/artifacts/obj/L02.Syntax/debug/`.

**.NET 10 adds file-based apps: one `.cs` file, no project.** `dotnet app.cs` (or `dotnet run app.cs`) generates a virtual project, builds it and runs it. `#:package`, `#:project`, `#:property` and `#:sdk` directives replace the project file — SDK 10.0.300 added `#:include` for extra source files — and `dotnet project convert` turns the file into a real project when it grows (File-based apps). It is the .NET answer to a Python or Kotlin script, with one trap: it inherits every `Directory.Build.props` above it. Inside this repository an unused variable in a file-based app became build error CS0219, and its build output moved into `.build/artifacts/`.

## 2.1 · Syntax map — Kotlin, Java and TypeScript → C#

| You already know | In .NET | Kind | Watch for |
|---|---|---|---|
| `package` + folders | `namespace X;` (file-scoped) | • Renamed | Not tied to folders; matching them is convention only |
| `fun main()` · `static void main` | Top-level statements | • Same idea | One file per project; the compiler generates the class |
| `import` | `using` · `global using` · implicit usings | • Renamed | `using` also means dispose — two unrelated uses of one keyword |
| Kotlin `val` · Java `final var` | `var` (mutable) · `const` · `readonly` field | • Trap | No `val`: a run-time value cannot be held in a read-only local; `var` is only inference |
| `BigDecimal` | `decimal` (struct, literal `0.07m`) | • Different | Operators work; 28–29 digits, not arbitrary precision |
| Kotlin `"$x"` · TS `` `${x}` `` | `$"{x:N2}"` · raw `"""` literals | • Renamed | A format specifier and the current culture apply inside the hole |
| Java `==` on String (reference) | `==` compares string content | • Different | `==` is ordinal; `IndexOf(string)`, `StartsWith`, `Compare` are linguistic |
| Kotlin `String?` | `string?` (nullable reference type) | • Trap | Compile-time warnings only — nothing checks at run time |
| `?.` `?:` `!!` | `?.` `??` `??=` `!` | • Trap | `!` never throws; `!!` does |
| Kotlin `when` · Java 21 `switch` | `switch` expression + patterns | • Different | Enum switches still need `_`: an enum is an int |
| Kotlin default + named args | Optional + named arguments | • Different | The default value is compiled into the caller |
| `vararg` · `...rest` | `params` any collection (C# 13) | • Renamed | `params ReadOnlySpan<T>` avoids the array |
| `Pair` · destructuring | Value tuples `(decimal Bonus, string Reason)` | • Renamed | Element names vanish at run time: reflection sees `Item1`, `Item2` |
| Checked exceptions, `throws` | Unchecked only, `catch … when` | • Different | A filter runs before inner `finally` blocks |
| try-with-resources · Kotlin `use` | `using var` · `await using` | • Renamed | Disposed at the end of the scope, in reverse order |

## 2.2 · An entry point — Kotlin main() vs C# top-level statements

**Kotlin — for comparison** — the idea you already know

```kotlin
// Main.kt: package, imports, a top-level main()
package motorquote.syntax

import java.util.Locale

fun main() {
    Locale.setDefault(Locale.ROOT)
    println("L02 - Kotlin in MotorQuote terms")
    TypesTour.run()
    StringsTour.run()
    NullsTour.run()
    PatternsTour.run()
    MethodsTour.run()
    ErrorsTour.run()
}
// Kotlin still emits a class (MainKt) around main
```

**C#** — samples/L02.Syntax/Program.cs · #region top-level

```csharp
// Program.cs IS the entry point: no class, no Main.
// Imports: GlobalUsings.cs + the SDK's implicit usings.
CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;

Console.WriteLine("L02.Syntax - C# in MotorQuote terms");
TypesTour.Run();
StringsTour.Run();
NullsTour.Run();
PatternsTour.Run();
MethodsTour.Run();
ErrorsTour.Run();
await ErrorsTour.RunAsync();   // await works at top level

return 0;                      // ...and so do args and return
```

**The same idea with less text** ( • Same idea ). The C# file has no `using` lines because `GlobalUsings.cs` and the SDK supply them. Top-level code may use `await`, `args` and `return`; the compiler generates the class and entry-point method around it. Only one file in a project may do this. The first statement pins the current culture to invariant so that every captured output in this lesson is reproducible — a demo choice: a service passes a culture or format provider to each call instead of relying on a process-wide default (§3).

## 2.3 · Two more files — global usings, and a namespace with no imports

C# samples/L02.Syntax/GlobalUsings.cs

```
// global using (C# 10): one line imports a namespace
// into every file of this project. With ImplicitUsings
// on, the SDK writes the same kind of line for System,
// System.IO, System.Linq ... into a generated file under
// obj/: L02.Syntax.GlobalUsings.g.cs.
global using L02.Rating;
global using L02.Syntax;   // for Program.cs: no namespace
```

C# samples/L02.Syntax/Resources.cs · #region file-scoped

```
namespace L02.Syntax;   // file-scoped (C# 10): no braces

// No using lines: IDisposable, ValueTask and Console
// resolve through the SDK's implicit global usings.

// a primary constructor (C# 12) - lesson 03
sealed class AuditScope(string name) : IDisposable
{
    public void Dispose() =>
        Console.WriteLine($"dispose {name}");
}

sealed class PartnerConnection : IAsyncDisposable
{
    public ValueTask DisposeAsync()
    {
        Console.WriteLine("async dispose partner connection");
        return ValueTask.CompletedTask;
    }
}
```

**Imports live in one place.** `GlobalUsings.cs` is the whole file: two lines that reach every file of `L02.Syntax`, including `Program.cs`. `Resources.cs` is also the whole file: `namespace L02.Syntax;` ends in a semicolon and covers everything below it, and `IDisposable`, `ValueTask` and `Console` need no `using` line because the SDK's implicit usings bring in `System` and `System.Threading.Tasks`. The two types are the resources section 7 disposes.

## 2.4 · A file-based app — the whole program, no project file

C# samples/file-based/premium-check.cs

```
#!/usr/bin/env -S dotnet --
#:project ../L02.Rating/L02.Rating.csproj
#:property PublishAot=false

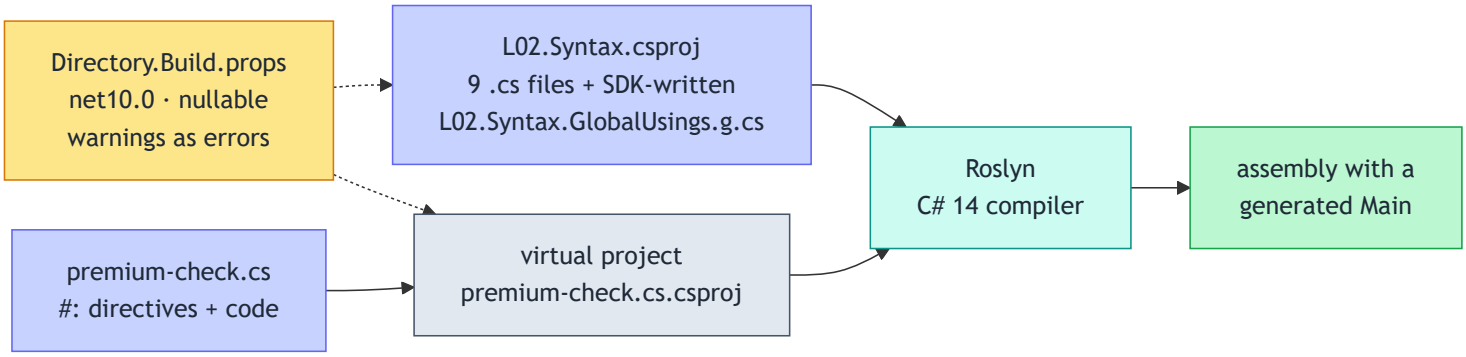
// A file-based app (.NET 10 SDK): no .csproj, no Main, no namespace.
//   dotnet premium-check.cs              the worked example
//   dotnet premium-check.cs -- 3         the same driver with 3 claims
using System.Globalization;
using L02.Rating;

var inv = CultureInfo.InvariantCulture;   // the same digits on every machine
int claims = args is [var first, ..] ? int.Parse(first, inv) : 0;
var request = Examples.YoungDriver(claims);

if (PremiumCalculator.TryCalculate(request, out var premium, out var reason))
    Console.WriteLine(string.Create(inv, $"total {premium.Total:N2} THB (net {premium.Net:N2})"));
else
    Console.WriteLine($"declined: {reason}");
```

**Directives first, code after.** `#:project` references the rating library exactly as a `<ProjectReference>` would; `#:property` sets an MSBuild property — here `PublishAot=false` turns off the Native AOT publish that file-based apps default to. `args is [var first, ..]` is a list pattern — 5.5. The app passes the invariant culture to each call, the per-call form, so `dotnet premium-check.cs` printed `total 8,933.71 THB (net 8,316.00)` and `dotnet premium-check.cs -- 3` printed `declined: 3+ claims in 5 years` whatever the machine's culture.

## 2.5 · What the compiler is given — a project build and a file-based run




indigo = your source · slate = generated by the SDK · amber = inherited build settings · teal = the compiler · green = output · dotted arrow = imported by MSBuild

## 2.6 · Date a codebase by its syntax — this lesson's C# by version

| C# · released | Syntax you meet in this lesson | Shown in |
|---|---|---|
| **6** · Jul 2015 | `$"…"` interpolation · `?.` · `nameof` · exception filters | 3.5 · 4.1 · 4.2 · 7.1 |
| **7.0–7.3** · 2017–18 | tuples and deconstruction · `out var` · `throw` expressions · declaration patterns · `1_200` · `in` parameters | 3.3 · 5.1 · 5.4 · 6.2 · 6.3 |
| **8** · Sep 2019 | nullable reference types · switch expressions · property and tuple patterns · `??=` · `using` declarations · static local functions | §4 · 5.1 · 5.3 · 6.3 · 7.1 · 7.2 |
| **9** · Nov 2020 | top-level statements · relational and logical patterns (`< 25`, `or`, `not`) | 2.2 · 5.1 · 5.3 |
| **10** · Nov 2021 | file-scoped namespaces · `global using` · extended property patterns | 2.3 · 5.4 |
| **11** · Nov 2022 | raw string literals · list patterns | 3.5 · 5.5 |
| **12** · Nov 2023 | collection expressions `[…]` · primary constructors | 2.3 · 5.5 |
| **13** · Nov 2024 | `params` collections | 6.2 |
| **14** · Nov 2025 | null-conditional assignment · modifiers on untyped lambda parameters | 4.1 · 6.3 |

## 3 · Types and values — decimal, strings and culture

**The built-in types are the JVM's with better names, plus one that matters to you: `decimal`.** `int` is an alias for `System.Int32` — a struct, so `List<int>` holds raw integers and there is no `Integer` wrapper to unbox. `decimal` is a 16-byte base-10 number with 28–29 significant digits (Floating-point numeric types): a value type with literals and operators, where Java makes you call `BigDecimal.multiply`. It represents 0.1 exactly; `double` does not, and the two never convert implicitly. Rounding is a default too: `Math.Round` sends a midpoint to the even neighbour unless you pass a `MidpointRounding` (Math.Round), where `BigDecimal.setScale` makes you name a mode.

**Integer overflow is silent by default.** Non-constant integral arithmetic runs in an `unchecked` context unless the `CheckForOverflowUnderflow` option is set, so with `int big = int.MaxValue`, `big + 1` wraps as in Java; `checked(…)` turns it into an `OverflowException`. The constant `int.MaxValue + 1` written directly does not compile at all (CS0220): constant expressions are checked. `decimal` throws on overflow in both contexts (checked and unchecked). Visual Basic makes the opposite default choice — 8.3.

**Strings are where a rating service quietly breaks.** Interpolated strings format numbers and dates with the *current culture*: the same premium prints as `8,933.71` or `8.933,71`, and a JSON payload built by interpolation changes shape. `IndexOf(string)`, `StartsWith(string)` and `Compare` are linguistic by default, while `==`, `Equals` and `Contains` are ordinal (Best practices for comparing strings); since .NET 5 the linguistic rules come from ICU on current Windows as well as on Linux (Globalization and ICU). And `th-TH` formats dates in the Thai Buddhist calendar — 2026 prints as 2569.

## 3.1 · Money — Java BigDecimal vs C# decimal

**Java — for comparison** — the idea you already know

```java
// Java: BigDecimal is a class — no literals, no operators
BigDecimal sumInsured = new BigDecimal("550000");
BigDecimal rate = new BigDecimal("0.021");
BigDecimal basePremium = sumInsured.multiply(rate);

double d = 0;
BigDecimal m = BigDecimal.ZERO;
for (int i = 0; i < 10; i++) {
    d += 0.1;
    m = m.add(new BigDecimal("0.1"));
}
System.out.println(d);             // 0.9999999999999999
System.out.println(m);             // 1.0

// no implicit double -> BigDecimal either
BigDecimal vat = basePremium.multiply(
    BigDecimal.valueOf(0.07));
```

**C#** — samples/L02.Syntax/TypesTour.cs · #region money

```csharp
decimal sumInsured = 550_000m;       // m suffix = decimal
decimal rate = 0.021m;
decimal basePremium = sumInsured * rate;

double d = 0;
decimal m = 0;
for (var i = 0; i < 10; i++)
{
    d += 0.1;
    m += 0.1m;
}
Console.WriteLine(d);                // 0.9999999999999999
Console.WriteLine(m);                // 1.0

// no implicit double -> decimal: say it
// (the tariff's VAT is on net + stamp duty, 9.4)
decimal vatOnBase = basePremium * (decimal)0.07;
Console.WriteLine($"{basePremium} {vatOnBase}");
```

**The same precision rule with far less ceremony** ( • Same idea ). `decimal` is a value type with a literal suffix (`m`) and ordinary operators. The sample printed `11550.000 808.50000`: a decimal keeps the scale of its operands, like `BigDecimal`. Format with `:N2` or round explicitly before you show or store a premium.

## 3.2 · Rounding — Java's explicit mode vs C#'s silent default

**Java — for comparison** — the idea you already know

```java
// Java: BigDecimal makes you name the mode
BigDecimal duty = new BigDecimal("2.345");
duty.setScale(2, RoundingMode.HALF_UP);    // 2.35
duty.setScale(2, RoundingMode.HALF_EVEN);  // 2.34
duty.setScale(2);  // ArithmeticException: rounding needed

// Math.round on a double: ties go toward +infinity
Math.round(2.5);                           // 3
```

**C#** — samples/L02.Rating/RatingRules.cs · #region round

```csharp
// Math.Round defaults to banker's rounding (ToEven):
//   Math.Round(2.345m, 2) -> 2.34
// A premium rounds half away from zero:
//   Round(2.345m)         -> 2.35
public static decimal Round(decimal amount) =>
    Math.Round(amount, 2, MidpointRounding.AwayFromZero);
```

**The C# default is the one a Java engineer does not expect** ( • Trap ). `Math.Round` without a mode is banker's rounding, `MidpointRounding.ToEven` (Math.Round): a satang at the midpoint goes to the even digit, so a premium can come out one satang lower than a HALF_UP system computes. The rating library states its rule in one place, and the passing test `Round_is_half_away_from_zero_not_bankers` asserts both results. Calculate (9.4) rounds every step through it.

## 3.3 · Aliases, var, const and overflow

**C#** — samples/L02.Syntax/TypesTour.cs · #region numbers

```csharp
int engineCc = 1_200;              // int is an alias of System.Int32
Int32 sameType = engineCc;         // the same struct: no wrapper, no boxing
var year = 2024;                   // var: inferred at compile time, still static
const int MaxClaims = 2;           // const: a compile-time literal
Console.WriteLine($"{typeof(int).FullName} {sameType} {year} {MaxClaims}");

int big = int.MaxValue;
Console.WriteLine(big + 1);      // unchecked: wraps
// the constant int.MaxValue + 1: compile error CS0220
try
{
    Console.WriteLine(checked(big + 1));
}
catch (OverflowException)
{
    Console.WriteLine("checked: OverflowException");
}
```

**Printed:** `System.Int32 1200 2024 2`, then `-2147483648` for `big + 1`, then `checked: OverflowException`. `var` is inference, not immutability — the variable can be reassigned. A `const` is copied into every assembly that reads it, so change a public `const` and its callers keep the old value until they are rebuilt; use `static readonly` for values that may change.

## 3.4 · The built-in types you will use every day

value view

### 1 int · long · short · byte

C# · Java/Kotlin · VB · • Same idea

Fixed-size integers, each an alias of a System struct (`int` = `System.Int32`).

| | |
|---|---|
| KOTLIN | Int, Long — the same sizes |
| DIFFERS | `byte` is unsigned (Java's is signed; C# has `sbyte`); `uint` and `ulong` exist |
| OVERFLOW | Wraps silently unless `checked` — 3.3; VB throws by default — 8.3 |
| LITERAL | `1_200` · `9_000_000_000L` · `0xFF` |

### 2 decimal — the money type

C# · • Different

128-bit base-10 with a scale: 0.1 is exact, and operators work.

| | |
|---|---|
| JAVA | BigDecimal — but a struct, with `+ - * /` and an `m` suffix |
| RANGE | 28–29 significant digits · 16 bytes · overflow always throws |
| SCALE | Keeps trailing zeros: 550000m × 0.021m prints `11550.000` |
| TRAP | Not a valid attribute argument — xUnit data needs a workaround, §9 |

### 3 double · float

C# · Java/Kotlin · • Same idea

IEEE 754 binary floating point, exactly as on the JVM.

| | |
|---|---|
| MEASURED | Adding 0.1 ten times prints `0.9999999999999999` |
| MIXING | No implicit conversion to or from `decimal` — cast, visibly |
| USE FOR | Ratios, telemetry, scores — never a premium |

## 3.4 · The built-in types (continued)

value view

### 4 string

C# · Java/Kotlin · • Different

Immutable UTF-16 text; alias of `System.String`.

| | |
|---|---|
| JAVA | == compares content in C#, not references |
| CULTURE | `IndexOf(string)`, `StartsWith(string)` and `Compare` are culture-sensitive by default |
| LITERALS | `$"…"` interpolation · `@"C:\x"` verbatim · `"""…"""` raw (C# 11) |

### 5 bool · char

C# · TypeScript · • Different

True/false, and one UTF-16 code unit.

| | |
|---|---|
| TS | No truthiness: `if (count)` does not compile |
| JAVA | boolean and char — identical semantics |

### 6 DateOnly · DateTime · DateTimeOffset

C# · Java/Kotlin · • Trap

A calendar date, a date and time, and a date and time with a UTC offset.

| | |
|---|---|
| JAVA | LocalDate · LocalDateTime · OffsetDateTime |
| TRAP | `th-TH` formats the Buddhist-era year: 1 Oct 2026 → `1/10/2569` (measured, 3.6) |
| RULE | Persist ISO 8601 with `CultureInfo.InvariantCulture` |

### 3.5 · Interpolation, verbatim and raw string literals

C# — samples/L02.Syntax/StringsTour.cs · #region strings

```
var make = "Toyota";
decimal total = 8_933.71m;              // the worked example's total
var de = CultureInfo.GetCultureInfo("de-DE");

// interpolation + format specifiers; the CURRENT culture picks the separators
Console.WriteLine($"{make} total {total:N2} THB");
Console.WriteLine(string.Create(de, $"{make} total {total:N2} THB"));

// verbatim string: backslashes are literal
var path = @"C:\quotes\2026\Q-0001.json";

// raw string literal (C# 11): no escaping; $$ means holes are {{ }}
var json = $$"""
    { "make": "{{make}}", "total": {{total}} }
    """;
Console.WriteLine(path);
Console.WriteLine(json);
```

**The culture is part of the output.** The same interpolation printed `Toyota total 8,933.71 THB` with the invariant culture and `Toyota total 8.933,71 THB` through `string.Create(de, …)`. The raw literal needs no escaped quotes; `$$` makes `{{ }}` the hole marker so single braces stay literal JSON. Build real JSON with `System.Text.Json`, not interpolation.

### 3.6 · Comparison and culture — the traps with measured output

C# — samples/L02.Syntax/StringsTour.cs · #region comparison

```
var tr = CultureInfo.GetCultureInfo("tr-TR");
var th = CultureInfo.GetCultureInfo("th-TH");

// culture-aware casing: Turkish upper-cases i to a dotted capital I
Console.WriteLine("file".ToUpper(tr));                          // FİLE
Console.WriteLine("file".ToUpperInvariant() == "FILE");         // True

// IndexOf(string) is linguistic by default and skips ignorable characters
char softHyphen = (char)0xAD;
string partnerCode = "MQ" + softHyphen + "TH";
Console.WriteLine(partnerCode.IndexOf("MQTH"));                 // 0
Console.WriteLine(partnerCode.IndexOf("MQTH", StringComparison.Ordinal)); // -1

// identifiers: compare ordinally, and say so
Console.WriteLine(string.Equals("mq-th", "MQ-TH", StringComparison.OrdinalIgnoreCase));

// th-TH formats dates in the Thai Buddhist calendar
var start = new DateOnly(2026, 10, 1);
Console.WriteLine(start.ToString("d/M/yyyy", th));              // 1/10/2569
Console.WriteLine(start.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture));
```

**Printed:** `FİLE`, `True`, `0`, `-1`, `True`, `1/10/2569`, `2026-10-01`. The linguistic `IndexOf` found `"MQTH"` inside a partner code that contains an invisible soft hyphen; the ordinal search did not. For identifiers, keys and protocol text, pass `StringComparison.Ordinal` or `OrdinalIgnoreCase` every time. Analyzers CA1307, CA1309 and CA1310 find the call sites that do not, but they are off by default — Microsoft documents turning them on with `AnalysisMode All`.

## 4 · Nullability — annotations, not a type system

**C# nullable reference types look like Kotlin's null safety and are enforced like TypeScript's `strictNullChecks`: by the compiler only.** With `<Nullable>enable` (set by current templates, and in this repository), `string` means "should not be null" and `string?` means "may be null". The compiler tracks a *null-state* — not-null or maybe-null — through assignments and checks, and warns. The runtime behaviour of the program is unchanged: `string` and `string?` are both `System.String` (Nullable reference types).

**Value types are different: `int?` is a real type.** It compiles to `Nullable<int>`, a struct with `HasValue` and `Value`, and boxing a null `int?` gives a null reference. So the same `?` is a run-time wrapper on a value type and a compile-time comment on a reference type.

**Null still gets in wherever the compiler cannot see.** `!` — the null-forgiving operator — only silences the warning for that one expression (the variable stays maybe-null); unlike Kotlin's `!!` it never throws. Deserializers and reflection ignore annotations: `System.Text.Json` stores a JSON `null` into a non-nullable property unless you opt in to

`RespectNullableAnnotations`, which arrived in .NET 9 and is off by default (Respect nullable annotations). For a quote API fed by ~30 external partners, validate at the boundary and treat annotations as documentation that the compiler checks.

**C# 14 closes one ergonomic gap.** `?.` may now appear on the left of an assignment: `draft?.Note = …` assigns only when `draft` is not null, and the right side is not evaluated otherwise; compound assignment works, `++` and `--` do not (What's new in C# 14).

### 4.1 · The null operators — Kotlin vs C#

**Kotlin — for comparison** — the idea you already know

```kotlin
val note: String? = lookUpNote("Q-0002")  // maybe-null
val length = note?.length ?: 0            // ?. then ?:
val shown = note ?: "no note"             // no ??=
println("$shown ($length)")

val sure = lookUpNote("Q-0001")!!         // THROWS if null
println(sure.uppercase())

val draft: QuoteDraft? = findDraft("Q-0001")
draft?.note = "young driver"           // safe-call assign
draft?.let { it.loading += BigDecimal("0.20") }
println("${draft?.note} ${draft?.loading}")
```

**C#** — samples/L02.Syntax/NullsTour.cs · #region nulls

```csharp
string? note = LookUpNote("Q-0002");  // maybe-null
int length = note?.Length ?? 0;       // ?. then ??
note ??= "no note";                   // assign only if null
Console.WriteLine($"{note} ({length})");

string sure = LookUpNote("Q-0001")!;  // ! = trust me
Console.WriteLine(sure.ToUpperInvariant());

QuoteDraft? draft = FindDraft("Q-0001");
draft?.Note = "young driver";         // C# 14: ?. on the left
draft?.Loading += 0.20m;              // compound works too
Console.WriteLine($"{draft?.Note} {draft?.Loading}");
```

**`?.` is the same** ( • Same idea ), **`??` is Elvis renamed** ( • Renamed ), **and `!` is a trap** ( • Trap ). `??=` assigns only when null and has no Kotlin twin; C# 14's `draft?.Note = …` matches Kotlin's safe-call assignment. Kotlin's `!!` is a run-time check that throws; C#'s `!` emits nothing. The sample printed `no note (0)`, `RENEWAL` and `young driver 0.20`.

### 4.2 · What survives to run time

**C#** — samples/L02.Syntax/NullsTour.cs · #region runtime

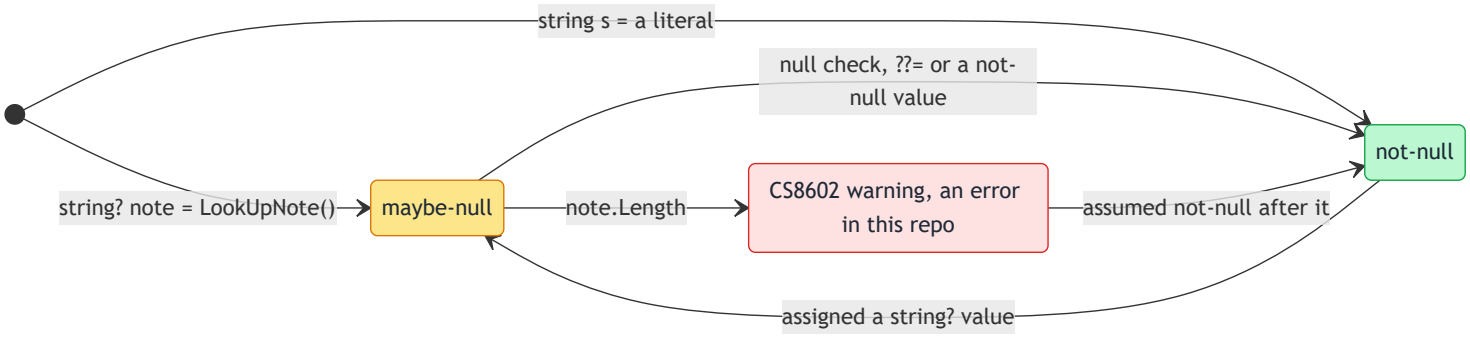
```csharp
// int? is a real run-time type: Nullable<int>, a struct with HasValue
int? claims = null;
Console.WriteLine(typeof(int?) == typeof(Nullable<int>));    // True
Console.WriteLine(claims.GetValueOrDefault());               // 0
object boxed = claims!;
Console.WriteLine(boxed is null);                            // True: boxing a null int? gives null

// string? is NOT a type: it is a compile-time annotation on System.String
var noteType = typeof(QuoteDraft).GetProperty(nameof(QuoteDraft.Note))!.PropertyType;
Console.WriteLine(noteType == typeof(string));               // True

// nothing enforces it at run time: a deserializer happily stores null
var dto = JsonSerializer.Deserialize<PartnerDto>("""{ "Name": null }""")!;
Console.WriteLine(dto.Name is null);                         // True
var strict = new JsonSerializerOptions { RespectNullableAnnotations = true };
try
{
    JsonSerializer.Deserialize<PartnerDto>("""{ "Name": null }""", strict);
}
catch (JsonException)
{
    Console.WriteLine("RespectNullableAnnotations: JsonException");
}
```

**Printed:** `True`, `0`, `True`, `True`, `True`, then `RespectNullableAnnotations: JsonException`. The non-nullable `Name` held `null` after default deserialization, with no warning anywhere in the code. The option covers explicit `null` values on non-generic members only — a missing property still leaves the default.

## 4.3 · Null-state analysis — what the compiler tracks for one variable




amber = maybe-null · green = not-null · red = the diagnostic you get · note!.Length silences the warning for that expression only, the variable stays maybe-null · the analysis does not look inside the methods you call

## 4.4 · Kotlin guarantees vs C# promises

**Kotlin guarantees — at run time too**

- `String` and `String?` are different types to the compiler
- The hole: Java platform types (`String!`), the twin of C#'s oblivious libraries
- `!!` throws `NullPointerException` at that line
- Parameters of public functions get generated null checks

**C# promises — at compile time only**

- `string` and `string?` are the same `System.String`
- Unannotated libraries are "oblivious" — no warnings either way
- `!` suppresses a warning and generates no code
- Guard public entry points yourself: `ArgumentNullException.ThrowIfNull(x)`

## 5 · Control flow and pattern matching

**The switch expression is Kotlin's `when` with a richer pattern language.** An input goes on the left, arms are `pattern => value`, the first match wins, and the whole thing is an expression. Patterns compose: relational (`< 25`), logical (`3 or 4`, `is not null`), property (`{ LicenceYears: >= 5 }`), positional and tuple (`(Commercial, > 3_000)`) and list patterns (`["RATE", var cls, ..]`) — the reference lists ten kinds (Patterns).

**This is the rating-rule notation you want.** A tariff table maps almost line for line onto arms, and the samples' age, claims, no-claim-bonus and commercial-use rules are each one switch. Java 21 finalised pattern switch with `when` guards (JEP 441) and record patterns (JEP 440); Kotlin 2.2 made guard conditions in `when` stable (What's new in Kotlin 2.2). C# goes further on property, relational and list patterns.

**The trap is exhaustiveness.** Kotlin proves a `when` over an enum complete. A C# enum is an integer with names: `(CoverageClass)7` compiles, so a switch over an enum without a discard arm draws a compiler warning — an error in this repository — and would throw `SwitchExpressionException` at run time. Every rule therefore ends in `_`, and the useful decision is what the discard does: return a default, or throw. C# 15, in preview with .NET 11, adds `closed` class hierarchies and `union` types whose switches the compiler does prove complete (What's new in C# 15); enums stay open.

## 5.1 · A rating rule — Kotlin when vs a C# switch expression

**Kotlin — for comparison** — the idea you already know

```kotlin
fun ageLoading(age: Int): BigDecimal = when {
    age < 18 -> error("under 18")    // throws
    age < 25 -> BigDecimal("0.20")   // young driver
    else -> BigDecimal.ZERO
}

// with a subject, a range reads like a tariff row:
// when (age) { in 18..24 -> BigDecimal("0.20") ... }
```

**C#** — samples/L02.Rating/RatingRules.cs · #region age-loading

```csharp
// relational patterns: one young-driver band
public static decimal AgeLoading(int age) => age switch
{
    < 18 => throw new NotSupportedException("under 18"),
    < 25 => 0.20m,
    _ => 0m,
};
```

**The same shape** ( • Same idea ). Relational patterns (C# 9) replace the repeated `age <`, and `throw` is allowed as an arm because it is an expression. Arms are checked top to bottom, so `< 25` only ever sees an age of 18 or more; the compiler reports an arm that can never match.

## 5.2 · The enum trap — a complete switch that still needs its discard arm

C# samples/L02.Rating/RatingRules.cs · #region base-rate

```
public static decimal BaseRate(CoverageClass coverage,
                               Money insured) =>
    coverage switch
    {
        CoverageClass.Class1 => insured.Amount * 0.021m,
        CoverageClass.Class2Plus => insured.Amount * 0.012m,
        CoverageClass.Class3Plus => insured.Amount * 0.009m,
        CoverageClass.Class3 => insured.Amount * 0.004m,
        // an enum is an int: (CoverageClass)7 lands here
        _ => throw new ArgumentOutOfRangeException(
            nameof(coverage)),
    };
```

C# samples/L02.Syntax/PatternsTour.cs · #region enum-trap

```
var bogus = (CoverageClass)7;        // compiles: an int
Console.WriteLine(Enum.IsDefined(bogus));   // False
try
{
    RatingRules.BaseRate(bogus, new Money(1m));
}
catch (ArgumentOutOfRangeException e)
{
    Console.WriteLine($"discard arm hit: {e.ParamName}");
}
```

**Four named values, five arms** ( • Trap ). `BaseRate` covers every `CoverageClass`, yet `(CoverageClass)7` compiles because an enum is an `int` with names. Without the `_` arm the compiler warns (an error in this repository) and the switch would throw `SwitchExpressionException`; with it, the rule chooses its own exception. **Printed:** `False`, then `discard arm hit: coverage`. Validate input with `Enum.IsDefined` before it reaches a rule.

## 5.3 · Property and tuple patterns in the rating rules

C# samples/L02.Rating/RatingRules.cs · #region rules

```
public static decimal ClaimsLoading(int claims)
{
    ArgumentOutOfRangeException.ThrowIfNegative(claims);
    return claims switch
    {
        0 => 0m,
        1 => 0.10m,
        2 => 0.25m,
        _ => throw new QuoteDeclinedException("3+ claims in 5 years"),
    };
}

// property patterns: test a record's properties, first match wins.
// A claim resets the ladder, so the guard arm comes first.
public static decimal NoClaimBonus(Driver driver) => driver switch
{
    { ClaimsLast5Years: > 0 } => 0m,
    { LicenceYears: >= 5 } => 0.50m,
    { LicenceYears: 4 } => 0.40m,
    { LicenceYears: 3 } => 0.30m,
    { LicenceYears: 2 } => 0.25m,
    { LicenceYears: 1 } => 0.20m,
    _ => 0m,
};

// a tuple pattern: switch on two values at once
public static decimal UseLoading(Vehicle vehicle) => (vehicle.Use, vehicle.EngineCc) switch
{
    (VehicleUse.Commercial, > 3_000) => 0.35m,
    (VehicleUse.Commercial, _) => 0.25m,
    _ => 0m,
};
```

**Three rules, three pattern styles.** `NoClaimBonus` tests a record's properties, `UseLoading` switches on a tuple of two values, and `ClaimsLoading` declines by throwing a domain exception. Tested by 15 of the 31 test cases.

### 5.4 · Type, property and declaration patterns on an object

C# — samples/L02.Syntax/PatternsTour.cs · #region patterns

```
object input = new Vehicle("Isuzu", "D-Max", 2023, 3_000,
                           VehicleUse.Commercial, new Money(900_000m));

// type pattern + property patterns (+ extended property pattern, C# 10)
string segment = input switch
{
    Vehicle { Use: VehicleUse.Commercial, EngineCc: >= 3_000 } => "heavy commercial",
    Vehicle { Use: VehicleUse.Commercial } => "commercial",
    Vehicle { SumInsured.Amount: > 2_000_000m } => "high value",
    Vehicle => "private",
    null => "nothing",
    _ => "not a vehicle",
};
Console.WriteLine(segment);

// `is` with a declaration pattern: test and narrow in one step
if (input is Vehicle { Year: < 2010 } old)
    Console.WriteLine($"old vehicle {old.Year}");
else if (input is not null)
    Console.WriteLine("recent or not a vehicle");
```

**Printed:** `heavy commercial`, then `recent or not a vehicle`. `SumInsured.Amount: > 2_000_000m` is an *extended property pattern* (C# 10) with a decimal constant. `input is Vehicle { Year: < 2010 } old` tests the type and a property, then binds the narrowed variable — the smart cast you know from Kotlin, with the variable named explicitly.

### 5.5 · List patterns — parsing a partner rate file

C# — samples/L02.Syntax/PatternsTour.cs · #region list-patterns

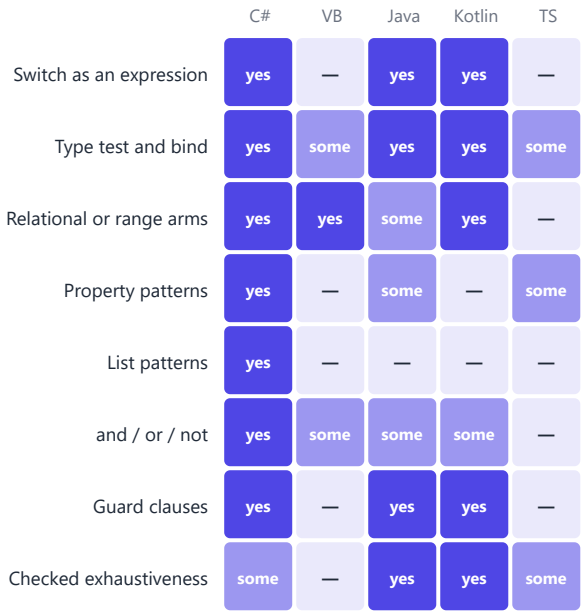
```
// a partner rate file: RATE,<class>,<percent>[,note]   (list patterns, C# 11)
string[] lines = ["RATE,Class1,2.1", "RATE,Class3", "RATE,Class2Plus,1.2,promo", "EOF"];
foreach (var line in lines)
{
    string parsed = line.Split(',') switch
    {
        ["RATE", var cls, var pct] => $"{cls} at {pct}%",
        ["RATE", var cls, var pct, .. var rest] => $"{cls} at {pct}% (+{rest.Length} field)",
        ["RATE", ..] => "rate line without a percent",
        ["EOF"] => "end of file",
        _ => "unknown line",
    };
    Console.WriteLine(parsed);
}
```

**Printed:** `Class1 at 2.1%`, `rate line without a percent`, `Class2Plus at 1.2% (+1 field)`, `end of file`. `..` is a slice and `.. var rest` captures it. The array itself is a collection expression (C# 12) — lesson 04 (Collections, LINQ & Functional C#) covers collections.

### 5.6 · Pattern kinds — the C# form, its version and your nearest equivalent

| Pattern | C# form | Since | Kotlin 2 · Java 21 |
|---|---|---|---|
| Constant | `0 => 0m, 1 => 0.10m` | C# 7.0 | `0 ->` · `case 0 ->` |
| Declaration / type | `input is Vehicle v` · `Vehicle => …` | C# 7.0 · type-only C# 9 | `is Vehicle` smart cast · `case Vehicle v` |
| Property | `{ LicenceYears: >= 5 }` · `{ A.B: 0 }` | C# 8 · extended C# 10 | none · record pattern (positional only) |
| Positional / tuple | `(VehicleUse.Commercial, > 3_000)` | C# 8 | none · `case Vehicle(var use, var cc)` |
| Relational | `< 25 => 0.20m` | C# 9 | `in 18..24 ->` · a `when` guard |
| Logical | `3 or 4` · `is not null` | C# 9 | `3, 4 ->` · `case 3, 4 ->` |
| List and slice | `["RATE", var cls, ..]` | C# 11 | none · none |
| var and discard | `var (x, y)` · `_ => 0m` | C# 7.0 · discard arm C# 8 | `else ->` · `default ->` |

## 5.7 · Pattern-matching support across the languages you read




C# 14 · VB · Java 21 · Kotlin 2.2 · TypeScript 5 · yes = first-class syntax · some = a subset, a statement form or an idiom · — = none · a rubric, not a benchmark

**C# has the widest pattern language; Java and Kotlin have the stronger exhaustiveness check.** C#'s "some" is the enum gap: a switch over an enum is never proven complete. VB's `Select Case` handles ranges well and little else. TypeScript's "some" is narrowing on `instanceof` or a discriminant property, plus the `never` exhaustiveness idiom. The cell values are the author's judgement ( • **Estimate** ).

## 6 · Methods and parameters

**C# method syntax is a superset of what Kotlin and TypeScript give you.** Named and optional arguments work as in Kotlin; `params` is varargs, and since C# 13 it accepts any collection type, including `ReadOnlySpan<T>`, so a call no longer has to allocate an array (What's new in C# 13). Expression-bodied members (`=>`) are Kotlin's single-expression functions. Local functions can be `static`, which makes the compiler refuse any accidental capture.

**C# can pass by reference, and for `ref` and `out` the call site has to say so** (`in` may be omitted). `ref` lets the callee write to the caller's variable, `out` requires the callee to assign it, and `in` passes a read-only reference. Neither Java nor Kotlin nor TypeScript has these; you will meet them in the `TryParse` pattern everywhere in the base library and in performance-sensitive code. Value tuples give you multiple return values with named elements and deconstruction — the answer to Kotlin's `Pair` and TypeScript's tuple types.

**Two versioning traps sit in the signature.** An optional parameter's default is copied into the *caller* at compile time, like a `const`: change it in a library and existing callers keep the old value until they recompile. And tuple element names do not exist at run time, so a public API that returns `(decimal, string)` is weaker than one that returns a small record — lesson 03 (Types, OOP & Generics).

## 6.1 · Parameter passing at a glance

| Form | What the callee gets | Caller writes | Nearest in your stack |
|---|---|---|---|
| (none) value type | A copy of the value | `Discount(13_860m)` | Java primitives |
| (none) reference type | A copy of the reference | `Calculate(request)` | Java / Kotlin / TS objects |
| `ref` | An alias of the caller's variable | `ref premium` | none — return the value or use a holder |
| `out` | An alias it must assign | `out var claims` | none — return a `Pair` or tuple |
| `in` | A read-only alias (avoids copying a large struct) | optional `in` | none |
| `params` | Any collection type (C# 13) | `Sum(8_316m, 33.26m)` | `vararg` · `...rest` |
| optional | The default, compiled into the caller | omit it, or name the next one | Kotlin and TS defaults, resolved in the callee |

## 6.2 · Declaring methods — TypeScript vs C#

**TypeScript — for comparison** — the idea you already know

```typescript
// TypeScript: defaults, rest parameters, tuples
function discount(amount: number, rate = 0.3,
                  roundUp = false): number {
  return roundUp ? Math.ceil(amount * rate)
                 : amount * rate;
}

function sum(...parts: number[]): number {
  return parts.reduce((a, b) => a + b, 0);
}

function bonus(d: Driver): [bonus: number, reason: string] {
  return d.claimsLast5Years === 0
    ? [noClaimBonus(d), "claim-free"]
    : [0, "has claims"];
}

// no by-reference parameters in TypeScript:
// return the new value instead
function load(premium: number, rate: number): number {
  return premium + premium * rate;
}
```

**C#** — samples/L02.Syntax/MethodsTour.cs · #region declarations

```csharp
// optional parameters, expression-bodied member.
// roundUp is a parameter demo: the tariff rounds half away
// from zero through RatingRules.Round (3.2), never up.
static decimal Discount(decimal amount, decimal rate = 0.40m,
                        bool roundUp = false) =>
    roundUp ? Math.Ceiling(amount * rate) : amount * rate;

// params collection (C# 13): a span, not an array
static decimal Sum(params ReadOnlySpan<decimal> parts)
{
    decimal total = 0;
    foreach (var part in parts) total += part;
    return total;
}

// a value tuple with named elements
static (decimal Bonus, string Reason) Bonus(Driver d) =>
    d.ClaimsLast5Years == 0
        ? (RatingRules.NoClaimBonus(d), "claim-free")
        : (0m, "has claims");

// ref: writes back to the caller's variable
// in:  passed by reference, read-only
static void Load(ref decimal premium, in decimal rate) =>
    premium += premium * rate;
```

**Defaults, rest and tuples are renamed** ( • Renamed ); **`ref` and `in` are different** ( • Different ). `params ReadOnlySpan<decimal>` takes arguments like `...parts` without allocating an array. Load changes the caller's variable in place — something TypeScript can only simulate by returning a value.

## 6.3 · Calling them — named arguments, deconstruction, out var and a C# 14 lambda

**C#** — samples/L02.Syntax/MethodsTour.cs · #region calls

```csharp
// named arguments: skip optional ones, any order
decimal d1 = Discount(13_860m, roundUp: true);
decimal d2 = Discount(rate: 0.20m, amount: 13_860m);

decimal total = Sum(8_316m, 33.26m, 584.45m);       // params span: no array allocated

var driver = Examples.YoungDriver().Driver;
var (bonus, reason) = Bonus(driver);                // deconstruct the tuple

decimal premium = 11_550m;
Load(ref premium, 0.20m);                           // ref must be repeated at the call site

if (int.TryParse("2", out var claims))              // out var: declared inline
    Console.WriteLine($"claims {claims}");

// a static local function: cannot capture locals by accident
static string Fmt(decimal x) => x.ToString("N2", CultureInfo.InvariantCulture);
Console.WriteLine($"{Fmt(d1)} {Fmt(d2)} {Fmt(total)} {bonus} {reason} {Fmt(premium)}");

// lambdas (lesson 04): C# 14 allows `out` on an untyped lambda parameter
TryParse<decimal> parsePercent = (text, out result) =>
    decimal.TryParse(text, CultureInfo.InvariantCulture, out result);
Console.WriteLine(parsePercent("2.1", out var pct) ? $"parsed {pct}" : "no");
```

**Printed:** `claims 2`, then `5,544.00 2,772.00 8,933.71 0.40 claim-free 13,860.00`, then `parsed 2.1`. Named arguments may skip optional parameters and change order. `ref` is repeated at the call site so a reviewer sees the write. In order the figures are the worked example's 40% no-claim discount on 13,860.00, the same call at 20% with named arguments, the total (9.5), the deconstructed `0.40 claim-free`, and the base after the `ref` loading; `roundUp` is there to show an optional `bool`, not because the tariff rounds up — it rounds half away from zero (3.2). C# 14 lets a lambda declare `out result` without spelling its type; lambdas and delegates belong to lesson 04 (Collections, LINQ & Functional C#).

## 7 · Exceptions and resources

**Every .NET exception is unchecked, so the signature tells you nothing.** There is no `throws` clause; a method that can decline a quote documents it with an XML `<exception>` tag and a test. Rethrow with `throw;` — `throw e;` restarts the stack trace, and analyzer rule CA2200, on as a warning by default, made it a build error in this repository. `throw` is also an expression, so it fits on the right of `??`, in a conditional, or as a switch arm (Exception-handling statements).

**Exception filters decide before anything unwinds.** `catch (E e) when (…)` evaluates its condition while the throwing frames are still on the stack — before any inner `finally` runs. A filter that returns `false` leaves the exception and its stack trace untouched, which makes `when (Log(e))` a way to log without catching. The sample proves the order; the sequence diagram shows why.

**`using` is try-with-resources without the block.** A *using declaration* (`using var x = …;`, C# 8) disposes at the end of the enclosing scope, and several declarations dispose in reverse order. `await using` does the same for `IAsyncDisposable` — a partner connection, a database transaction (using statement). To write a disposable type, a `sealed` class needs only a public `Dispose()`, as `AuditScope` in 2.3 shows; an unsealed class follows the dispose pattern — a `protected virtual Dispose(bool)` that subclasses override — and needs a finalizer only when it holds an unmanaged handle directly (Implement a Dispose method).

### 7.1 · Resources and filtered catch — Java vs C#

**Java — for comparison** — the idea you already know

```java
// Java: try-with-resources + a checked exception
try (var audit = new AuditScope("audit");
     var trace = new AuditScope("trace")) {
    var request = Examples.youngDriver(3);
    var premium = calculator.calculate(request);
    System.out.println(premium.total());
} catch (QuoteDeclinedException e) {
    if (!e.getReason().contains("claims")) {
        throw e;               // no filter: rethrow
    }
    System.out.println("declined: " + e.getReason());
}
// trace, then audit, are closed after the try
```

**C#** — samples/L02.Syntax/ErrorsTour.cs · #region resources

```csharp
using var audit = new AuditScope("audit");  // declaration
using var trace = new AuditScope("trace");  // disposed first

var request = Examples.YoungDriver(claims: 3);
try
{
    var premium = PremiumCalculator.Calculate(request);
    Console.WriteLine(premium.Total);
}
catch (QuoteDeclinedException e)
    when (e.Reason.Contains("claims"))  // filter
{
    Console.WriteLine($"declined: {e.Reason}");
}
// end of scope: trace, then audit, are disposed
```

**Renamed, with one real difference** ( • Renamed ). The Java resources close when the `try` block ends; the C# using declarations live until the end of the method. The filter replaces catch-check-rethrow. Printed: `declined: 3+ claims in 5 years`, `dispose trace`, `dispose audit`.

### 7.2 · When does a filter run? Proven by the sample

**C#** — samples/L02.Syntax/ErrorsTour.cs · #region filter-order

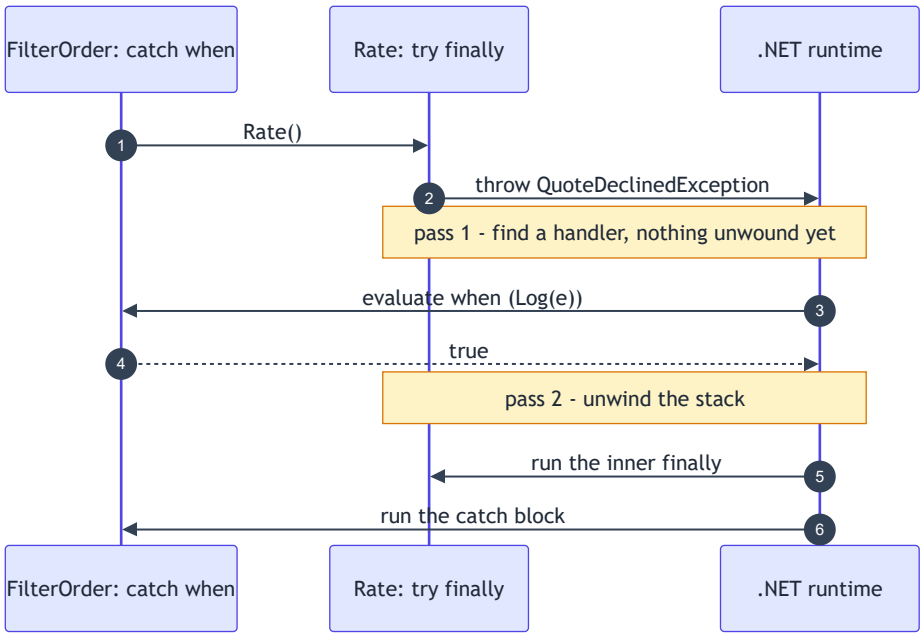
```csharp
try
{
    Rate();
}
catch (Exception e) when (Log(e))              // 1st: the filter runs BEFORE unwinding
{
    Console.WriteLine("catch block");         // 3rd
}

static void Rate()
{
    try { throw new QuoteDeclinedException("3+ claims in 5 years"); }
    finally { Console.WriteLine("inner finally"); }   // 2nd
}

static bool Log(Exception e)
{
    Console.WriteLine($"filter sees {e.GetType().Name}");
    return true;                               // false = keep searching up the stack
}
```

**Printed, in this order:** `filter sees QuoteDeclinedException`, `inner finally`, `catch block`. The filter ran while `Rate()` was still on the stack; only after it returned `true` did the runtime unwind — running the inner `finally` — and enter the catch block.

## 7.3 · Two-pass exception handling




pass 1 searches for a handler and runs filters with the stack intact · pass 2 unwinds, runs finally blocks, then the catch block

## 8 · Visual Basic alongside

**The C# language tour has a Visual Basic twin, and it reads like a verbose cousin rather than a stranger.** The twin covers types, nulls, branching, methods and errors — strings and culture are left out because they are the same framework calls. `Dim x As Decimal` declares, `Nothing` is null, `If(a, b)` is `??` and `If(c, a, b)` is the conditional operator — both short-circuit, unlike the legacy `IIf` function that evaluates all its arguments (VB If operator). `Select Case` handles ranges (`Case 18 To 24`, `Case Is < 18`); `Using … End Using` and `Catch … When` are the same resource scope and exception filter (VB Try...Catch...Finally).

**What VB lacks is the post-2015 syntax.** No nullable reference annotations (`String` is simply nullable), no switch expression or property and list patterns, no `??=`, no `?.` on the left of an assignment, no tuple deconstruction. The VB code reaches the same results with `If … Is Nothing` and statements — which is why a migration estate's VB rating code is longer, not wrong.

**One default is reversed, and it changes behaviour.** VB checks integer overflow unless compiled with `-removeintchecks` (-removeintchecks): the same `big + 1` that wraps silently in C# threw `OverflowException` in `L02.SyntaxVb` (8.3). Porting VB arithmetic to C# without `checked` removes a guard nobody wrote down. The rest of VB's semantics are the subject of lesson 06 (VB.NET for Legacy Estates).

## 8.1 · The age loading — C# switch expression vs VB Select Case

**C#** samples/L02.Rating/RatingRules.cs · #region age-loading

```
// relational patterns: one young-driver band
public static decimal AgeLoading(int age) => age switch
{
    < 18 => throw new NotSupportedException("under 18"),
    < 25 => 0.20m,
    _ => 0m,
};
```

**Visual Basic** samples/L02.SyntaxVb/Tour.vb · #region select-case

```
Function AgeLoading(age As Integer) As Decimal
    Select Case age
        Case Is < 18
            Throw New NotSupportedException("under 18")
        Case 18 To 24
            Return 0.2D
        Case Else
            Return 0D
    End Select
End Function
```

**Same band, same result.** `Select Case` is a statement, not an expression, so each arm is a `Case` line plus a `Return` or a `Throw`; `Case Is < 18` is its relational pattern and `Case 18 To 24` its range form — the same rule the C# switch writes as `< 25` below a `< 18` arm. The VB run printed age `23 loading 0.2`.

## 8.2 · Resources and a filtered catch — C# using var vs VB Using … End Using

**C#** samples/L02.Syntax/ErrorsTour.cs · #region resources

```csharp
using var audit = new AuditScope("audit");  // declaration
using var trace = new AuditScope("trace");  // disposed first

var request = Examples.YoungDriver(claims: 3);
try
{
    var premium = PremiumCalculator.Calculate(request);
    Console.WriteLine(premium.Total);
}
catch (QuoteDeclinedException e)
    when (e.Reason.Contains("claims"))  // filter
{
    Console.WriteLine($"declined: {e.Reason}");
}
// end of scope: trace, then audit, are disposed
```

**Visual Basic** samples/L02.SyntaxVb/Tour.vb · #region errors

```vb
Using audit As New AuditScope("audit"),
      trace As New AuditScope("trace")
    Try
        Dim request = Examples.YoungDriver(claims:=3)
        Dim premium = PremiumCalculator.Calculate(request)
        Console.WriteLine(premium.Total)
    Catch e As QuoteDeclinedException _
        When e.Reason.Contains("claims")
        Console.WriteLine($"declined: {e.Reason}")
    End Try
End Using                       ' trace, then audit, disposed
```

**A VB `Using` is a block, like Java's try-with-resources.** One statement declares both resources and disposes them in reverse order at `End Using`, where the C# using declarations live to the end of the method. `Catch … When` is the same filter, evaluated before anything unwinds (7.2). VB printed the C# lines of 7.1: `declined: 3+ claims in 5 years`, `dispose trace`, `dispose audit`.

## 8.3 · Overflow — the one default VB reverses

**C#** samples/L02.Syntax/TypesTour.cs · #region overflow

```csharp
int big = int.MaxValue;
Console.WriteLine(big + 1);      // unchecked: wraps
// the constant int.MaxValue + 1: compile error CS0220
try
{
    Console.WriteLine(checked(big + 1));
}
catch (OverflowException)
{
    Console.WriteLine("checked: OverflowException");
}
```

**Visual Basic** samples/L02.SyntaxVb/Tour.vb · #region overflow

```vb
Dim big = Integer.MaxValue
Try
    Console.WriteLine(big + 1)   ' checked by default
Catch ex As OverflowException
    Console.WriteLine("VB: OverflowException without checked")
End Try
' the constant Integer.MaxValue + 1: compile error BC30439
```

**The same `big + 1`, two behaviours** ( • Trap ). C# printed `-2147483648`; VB threw and printed `VB: OverflowException without checked`. Neither compiler accepts the constant form: written directly, `int.MaxValue + 1` is error CS0220 and `Integer.MaxValue + 1` is error BC30439 (both measured on the build machine).

## 8.4 · This lesson's C# in Visual Basic

| C# | Visual Basic | Watch for |
|---|---|---|
| `string?` · `int?` | `String` · `Integer?` | VB has no nullable reference annotations |
| `null` · `is null` · `is not null` | `Nothing` · `Is Nothing` · `IsNot Nothing` | `Nothing` assigned to `Integer` means 0 |
| `a ?? b` · `c ? a : b` | `If(a, b)` · `If(c, a, b)` | Never `IIf`: it evaluates both branches |
| `0.07m` · `(decimal)0.07` | `0.07D` · `CDec(0.07)` | VB's `D` suffix is Decimal, C#'s `d` is double — C# money needs `m`; Option Strict On makes VB write `CDec` |
| Unchecked `int` arithmetic | Checked by default | Porting to C# drops the guard |
| `switch` expression + patterns | `Select Case` with `Case Is` and `To` | A statement — no property or list patterns |
| `x is Vehicle v` | `TypeOf x Is Vehicle` + `DirectCast` | Two steps: test, then cast |
| `Discount(x, roundUp: true)` | `Discount(x, roundUp:=True)` | `:=` for named arguments |
| `ref premium` at the call | `ByRef` in the declaration only | A VB call site does not show the write |
| `using var a = …;` | `Using a As New … … End Using` | Several resources in one `Using`, disposed in reverse |
| `catch (E e) when (…)` | `Catch e As E When …` | Same two-pass semantics |

## 9 · Hands-on — run the tour, the tests and the file-based app

**4 projects and one file, all run from the repository root.** The tour prints in both languages, the rating rules meet their 31 test cases, and `premium-check.cs` runs with no project file at all.

**The tests met a decimal trap of their own.** An attribute argument must be a constant of an attribute parameter type, and `decimal` is not one: `[InlineData(24, 0.20m)]` fails with CS0182. The tests use `TheoryData<int, decimal>` for decimal data, integer percentages where that reads better, and ISO date strings parsed with the invariant culture for `DateOnly`. xUnit itself is the subject of lesson 10 (Testing, TDD & 100% Coverage).

**The worked example is the number to check your reading against.** A 2024 private car insured for 550,000 THB, Class 1, a 23-year-old driver with four claim-free licence years, starting 1 October 2026: 11,550.00 base, +20% young-driver loading, −40% no-claim bonus, 0.4% stamp duty and 7% VAT — **8,933.71 THB, illustrative, not a real tariff**.

**It is also the number to carry forward.** Every lesson of this track prices with this one tariff — the base rate per coverage class, the young-driver, claims and commercial-use loadings, the no-claim ladder of 0/20/25/30/40/50% by claim-free licence year, and the two rounded taxes — so a quote you re-rate in a later lesson comes out at the same satang. A lesson that varies it on purpose says so and names it: the legacy rounding and typing quirks of lesson 06 (VB.NET for Legacy Estates) are the deliberate case. The decline at three claims in five years is part of the tariff too; here it is a `QuoteDeclinedException` (§7), and a lesson that teaches data instead of exceptions records it as a declined status.

### 9.1 · Commands

**Terminal**

```
# from the repository root, .NET 10 SDK first on PATH
dotnet run --project lesson-02-csharp-language-essentials/samples/L02.Syntax
dotnet run --project lesson-02-csharp-language-essentials/samples/L02.SyntaxVb
dotnet test lesson-02-csharp-language-essentials/samples/L02.Rating.Tests
python 0-script/verify_samples.py --only 2    # every project: build, test, run
# the file-based app: no project file, arguments after --
cd lesson-02-csharp-language-essentials/samples/file-based
dotnet premium-check.cs
dotnet premium-check.cs -- 3
```

**No separate restore or build step:** `dotnet run` and `dotnet test` restore and build first. Everything after `--` goes to the app as `args`. `verify_samples.py` covers the projects, not the file-based app.

### 9.2 · What you should see — the C# and VB tours

**Output — L02.Syntax (first part)** captured on the build machine

```
L02.Syntax - C# in MotorQuote terms

[types]
System.Int32 1200 2024 2
-2147483648
checked: OverflowException
0.9999999999999999
1.0
11550.000 808.50000

[strings]
Toyota total 8,933.71 THB
Toyota total 8.933,71 THB
C:\quotes\2026\Q-0001.json
{ "make": "Toyota", "total": 8933.71 }
FİLE
True
0
-1
True
1/10/2569
2026-10-01
```

**Output — L02.SyntaxVb** captured on the build machine

```
L02.SyntaxVb - the same tour in Visual Basic

[types]
System.Int32 1200 2024 2
VB: OverflowException without checked
0.9999999999999999
1.0
11550.000 808.50000

[nulls]
no note (0)
RENEWAL
young driver 0.2
0

[patterns]
age 23 loading 0.2
Toyota 1200 cc

[methods]
5,544.00 8,933.71 0.40 claim-free 13,860.00

[errors]
declined: 3+ claims in 5 years
dispose trace
dispose audit
```

**Compare the line after `System.Int32 1200 2024 2` in each.** C# wrapped to `-2147483648`; VB threw. The double error, the decimal scale and the dispose order match across the two languages because both compile to the same runtime types — `System.Double`, `System.Decimal`, `IDisposable`; only the no-claim bonus and the declined quote come from the shared C# `L02.Rating` library. VB prints `0.2` where C# prints `0.20`

because its literal `0.2D` has one decimal place. The rest of the C# tour is quoted as *Printed* under each panel in §4–§7, and it ends with `async dispose partner connection` because the `await using` declaration lives until `RunAsync` returns.

## 9.3 · The test run and the file-based app

**Terminal — the test run and the file-based app** — captured on the build machine

```
> dotnet test lesson-02-csharp-language-essentials/samples/L02.Rating.Tests
# restore, build and xUnit progress lines left out
  L02.Rating.Tests test net10.0 succeeded (2.1s)
Test summary: total: 31, failed: 0, succeeded: 31, skipped: 0, duration: 2.1s
Build succeeded in 3.1s

> cd lesson-02-csharp-language-essentials/samples/file-based
> dotnet premium-check.cs
total 8,933.71 THB (net 8,316.00)
> dotnet premium-check.cs -- 3
declined: 3+ claims in 5 years
```

**31 test cases, all green, and the same total from a file with no project.** In an interactive terminal the .NET 10 SDK prints this compact *terminal logger* summary; redirected output falls back to the classic `Passed! - Failed: 0, Passed: 31` line. The file-based app calls the same `L02.Rating` library through `#:project`, so its 8,933.71 THB matches the worked example to the satang.

## 9.4 · The premium calculation — the code behind the worked example

**C#** — samples/L02.Rating/PremiumCalculator.cs · #region calculate

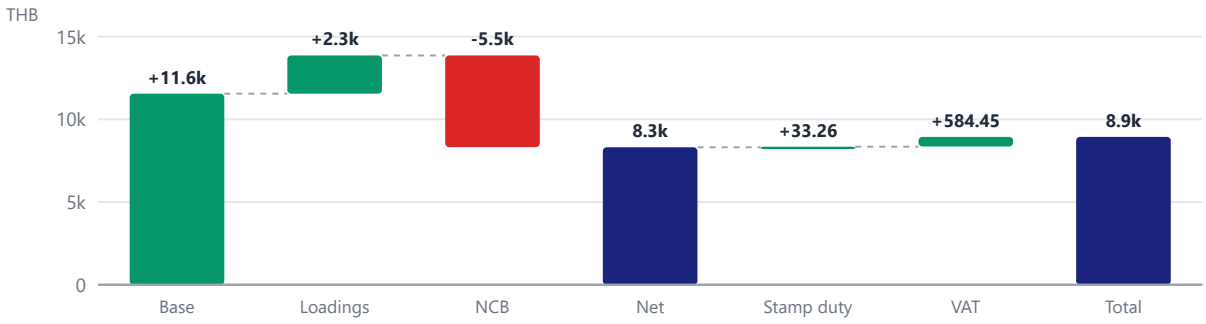
```csharp
public static Premium Calculate(QuoteRequest request)
{
    ArgumentNullException.ThrowIfNull(request);
    var (vehicle, driver, coverage, start) = request;   // positional deconstruction

    decimal basePremium = Round(BaseRate(coverage, vehicle.SumInsured));
    decimal loadingRate = AgeLoading(driver.AgeOn(start))
                        + ClaimsLoading(driver.ClaimsLast5Years)
                        + UseLoading(vehicle);
    decimal loadings = Round(basePremium * loadingRate);
    decimal discount = Round((basePremium + loadings) * NoClaimBonus(driver));
    decimal net = basePremium + loadings - discount;
    decimal stampDuty = Round(net * StampDutyRate);        // 0.4% of net
    decimal vat = Round((net + stampDuty) * VatRate);      // 7% of net + duty

    return new Premium(basePremium, loadings, discount, net,
                       stampDuty, vat, net + stampDuty + vat);
}
```

**The order of the statements is the tariff.** The rule names need no class prefix because the file opens with `using static L02.Rating.RatingRules;` (C# 6). Every money step goes through `Round` (3.2) before the next step reads it; the no-claim bonus applies to base plus loadings, and VAT to net plus stamp duty. `var (vehicle, driver, coverage, start) = request` deconstructs the record — records themselves are lesson 03 (Types, OOP & Generics).

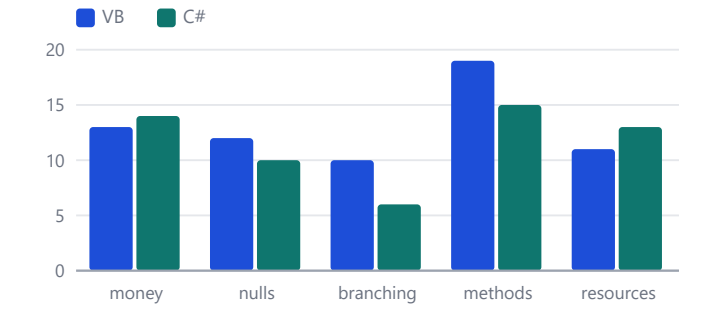
## 9.5 · The worked example — how 8,933.71 THB is built




illustrative tariff, not a real one · each figure read at build time from the Assert lines of the passing test Calculate_matches_the_worked_example · measured

**The no-claim bonus outweighs the young-driver loading.** Base 11,550.00, +2,310.00 for age 23, then −5,544.00 for four claim-free years, because the bonus applies to base plus loadings: net 8,316.00. Stamp duty (33.26) is barely visible; VAT (584.45) is charged on net plus duty, for a total of 8,933.71. The order of the rounding steps is part of the rule, and the test pins every one ( • **Measured** ).

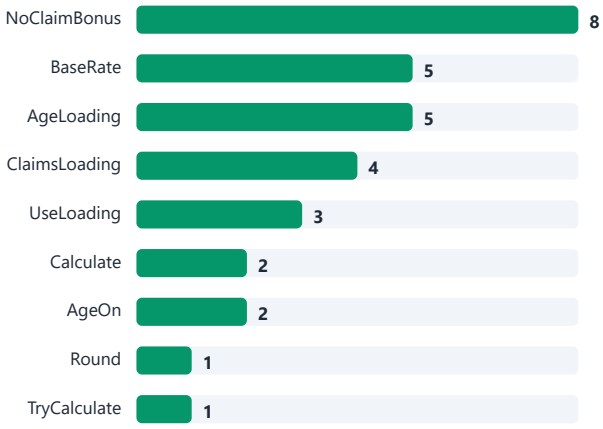
### 9.6 · The same behaviour, C# vs VB lines




non-blank, non-comment lines in matching #region pairs · measured at build

**VB's biggest overhead is methods: 19 lines against 15, because VB spells `Optional`, `ParamArray`, `ByRef` and `End Function`.** At the other end, resources takes 11 VB lines against 13 in C# — VB closes its blocks with `End Try` and `End Using` where C# spends four lines on braces. • Measured; a verbosity count, not a quality score.

### 9.7 · Test cases per tested member




[Fact] = 1 · [InlineData] or TheoryData row = 1 · counted at build

**The banded rules carry most of the cases.** Each age-band edge is a case: a < typed as <= changes a premium. • Measured.

### ⚠ Traps — they look right to a JVM, Kotlin or TypeScript engineer

1. **`!` is not `!!`.** It silences a warning and generates nothing; a null that arrives from JSON, reflection or an unannotated library flows on until something dereferences it far away.
2. **`var` is not `val`.** No local can hold a run-time value read-only: `const` takes only constants, `var` only infers the type.
3. **Culture leaks into strings and dates.** `IndexOf(string)` matched across a soft hyphen, `th-TH` printed 2569, `de-DE` put a comma in a premium: use `StringComparison.Ordinal` for identifiers, the invariant culture for stored data.
4. **`Math.Round` rounds half to even by default.** `Math.Round(2.345m, 2)` is 2.34 (Math.Round), asserted by a passing test (3.2). State the `MidpointRounding` of every premium calculation.
5. **An enum is an int.** `(CoverageClass)7` compiles: every enum switch needs `_`, and input parsing needs `Enum.IsDefined`.

### ✓ Checkpoint — you are ready for lesson 03 when

1. `python 0-script/verify_samples.py --only 2` reports 4/4 sample projects passed on your machine, with 31 test cases green, and `dotnet premium-check.cs` prints the 8,933.71 THB total.
2. You can explain every line of output in 9.2 — especially `0.9999999999999999`, `11550.000`, `FİLE` and `2569`.
3. You can rewrite a Kotlin `when` as a C# switch expression with relational, property and tuple patterns, and say what its discard arm does.
4. You can read the VB `Select Case`, `If()`, `Using` and `Catch … When` versions of the same rules.

### 🔍 Check yourself

1. Why does declaring a property as non-nullable `string` not stop `System.Text.Json` from storing null in it, and what .NET 9 option changes that?
2. With `int big = int.MaxValue`, what does `big + 1` evaluate to in C# and in Visual Basic with default settings — and why does `int.MaxValue + 1` written directly not compile?
3. Which of `==`, `Contains(string)` and `IndexOf(string)` is culture-sensitive by default?
4. A switch expression over `CoverageClass` handles all four named values. Why does the compiler still want a `_` arm?
5. In `catch (Exception e) when (Log(e))`, does `Log` run before or after the `finally` blocks of the method that threw?
6. A library changes an optional default from `0.40m` to `0.30m`. What does a caller that omits it pass until rebuilt, and why?
7. Which settings does a file-based app inside this repository inherit, and from which file?

**Lesson 02 in one line:** C# spells your Kotlin, Java and TypeScript habits with less ceremony — but `string?` is only a warning, `!` never throws, `decimal` is the money type, culture and banker's rounding are defaults you must override, and an enum switch always needs `_`.
**Next:** lesson 03 (Types, OOP & Generics) — classes, structs and records, and the generics that type erasure never gave you.